Cortex Code CLI

Quickstart for Data Engineering Workflows

What this guide covers

This quickstart walks through a realistic finance use case using Cortex Code CLI. You begin inside Snowflake to discover data, compare schemas, and build a Dynamic Table for AP invoices. You then add local business context with a reusable PRD Evaluator skill and, optionally, use the resulting data product as the foundation for a Cortex data agent.

Designed for hands-on use: run it end to end as a lab or reuse individual prompts and patterns in your own projects.

What This Quickstart Is For

This quickstart is written for data engineers and analytics engineers who already have AP-style source data in Snowflake and want a fast, realistic way to evaluate what Cortex Code CLI can do for day-to-day workflows. Rather than presenting a disconnected set of prompts, it follows a single AP invoices storyline from discovery through operationalization.

Audience	What you will be able to do
Data engineers and analytics engineers	Discover source tables, normalize SAP and Oracle invoice schemas, build a Silver Dynamic Table, operationalize it with a bundled skill, and extend the workflow with a reusable PRD Evaluator skill and optional agent design.

Before You Begin

Lab environment

Database	COCO_WORKSHOP
Shared inputs	COCO_WORKSHOP.SOURCE_DATA (read-only)
Your outputs	COCO_WORKSHOP.<YOUR_SCHEMA>
Role	COCO_WORKSHOP_ROLE
Warehouse	COCO_WORKSHOP_WH

Set your context before you start

```
USE ROLE COCO_WORKSHOP_ROLE;
USE WAREHOUSE COCO_WORKSHOP_WH;
USE DATABASE COCO_WORKSHOP;
USE SCHEMA <YOUR_SCHEMA>;
```

You also need: access to the objects used in this lab, and the SNOWFLAKE.CORTEX_USER database role on your user (directly or via a parent role) so Cortex Code CLI can use Snowflake AI features.

Install and connect

If Cortex Code CLI is already installed and working for your Snowflake account, you can skip this section. Otherwise, follow the official installation instructions in Snowflake documentation and then return here for the workflow steps. The quick path is below.

Scenario	Command
Linux, macOS, WSL	curl -LsS https://ai.snowflake.com/static/cc-scripts/install.sh | sh cortex --version
Windows (PowerShell)	irm https://ai.snowflake.com/static/cc-scripts/install.ps1 | iex cortex --version

Launch the CLI with **cortex**. The setup wizard will guide you through choosing or creating a Snowflake connection and validating account access. Cortex Code CLI can also reuse existing Snowflake CLI connections from ~/.snowflake/connections.toml (preferred) or ~/.snowflake/config.toml (legacy).

Optional named connection: cortex -c <your_connection_name>

Cortex Code CLI Core Concepts

Cortex Code CLI is Snowflake's AI coding agent in the terminal. It provides an agentic shell that understands both your Snowflake environment and your local project so you can work in natural language while staying in context.

Snowflake-aware shell

Cortex Code CLI connects to your existing Snowflake connections and respects roles, warehouses, databases, and schemas. It uses this context to generate and refine SQL, plans, and objects against the correct environment.

Skills (bundled and custom)

Cortex Code organizes Snowflake workflows into skills. Bundled skills encode Snowflake best practices for areas such as Dynamic Tables, semantic views, and agents. Custom skills live with your project and capture your team's repeatable patterns, such as how to interpret PRDs and structure change plans.

Safe, reviewable execution (modes)

Cortex Code CLI supports execution modes that make changes explainable and controlled: you can have it plan and present multi-step work before anything runs, require explicit confirmation for impactful operations, or opt into auto-execution only in trusted environments where you are comfortable letting the agent carry out approved workflows end-to-end.

Quickstart Path

Step	Demo	Outcome
1	**Demo 1**	Discover source tables, compare SAP and Oracle schemas, build a Silver-grade AP invoices Dynamic Table, and generate an operating runbook with the Dynamic Table skill.
2	**Demo 2**	Read a local PRD, create a reusable PRD Evaluator skill, and apply repeatable updates to SILVER_AP_INVOICES.
3	**Demo 3 (optional)**	Use the curated invoices object as the foundation for a Cortex data agent and establish a simple evaluation workflow.

Demo 1 – Start Inside Snowflake (20 min)

Value in this section: You begin with the highest-confidence path: data that is already in Snowflake. In this demo you discover the right source tables, compare SAP and Oracle schemas, generate a Silver-grade Dynamic Table with readable SQL, and use a bundled Dynamic Table skill to understand how the object should be operated and monitored.

Client story: Finance needs a trusted AP invoices layer. SAP and Oracle invoice data already exist in Snowflake, but there is no standardized Silver object yet. The goal is a single, well-modeled table that can be explained, monitored, and evolved without repeating discovery work each time a requirement changes.

Step 1.1 – Discover the source

Begin with data discovery. This is a core Cortex Code CLI workflow and a natural first move when you enter a new schema.

Prompt

What tables are in the `COCO_WORKSHOP.SOURCE_DATA` schema?
For each table, give me a one-line description of what it appears to contain and identify which ones are most relevant to an AP invoices Silver layer.

What to look for

- A short list of tables that matter for the AP workflow.
- Enough description to orient yourself without manually opening each object.

Step 1.2 – Compare SAP and Oracle invoice schemas

Once you know which tables matter, compare them and highlight differences. This gives you the shortest path to a clean normalization plan.

Prompt

Compare the columns between
COCO_WORKSHOP.SOURCE_DATA.BRONZE_SAP_AP_INVOICES and
COCO_WORKSHOP.SOURCE_DATA.BRONZE_ORACLE_AP_INVOICES.
Return:
- Equivalent fields with different names
- Fields that require type normalization or default values
- Differences that should remain open questions instead of becoming hidden assumptions

What to look for

- Equivalent business fields with different names.
- Normalization work you should expect before building Silver.
- Open questions that should stay explicit for later review.

Step 1.3 – Generate the Silver Dynamic Table

Convert the mapping work into a production-quality first pass. Ask Cortex Code CLI for readable SQL, explicit assumptions, and a structure you would be comfortable committing to your repo.

While this task is not overly complex, we are going to turn on Plan Mode in the CLI by holding down the keys CTRL-P to see how Cortex Code can think through complex tasks.

Hit Terminal Keys

Ctrl – P

What is Plan Mode?

Plan mode is one of a few different execution modes that Cortex Code CLI enables users to enter based on the task they are working on.

Mode	Description	Use Case	Activation
Interactive	Proposes changes and asks for confirmation before running impactful operations.	Everyday work where you want to see and approve each step.	Default
Plan mode	Stays read-only while it thinks, then returns a structured multi-step plan and waits for your approval before executing.	Multi-step or higher-risk tasks, such as creating or updating core tables.	Press `Ctrl+P` or enter `/plan` to turn plan mode on
Automated (trusted environments)	Executes an agreed workflow end to end with fewer prompts, once you are comfortable with the pattern.	Trusted, non-production or tightly controlled environments where the workflow has already been validated.	Use `Shift+Tab` to move into the more automated mode your team has approved for trusted environments.

Prompt

Use database COCO_WORKSHOP and my current schema for outputs.
Create a Dynamic Table called SILVER_AP_INVOICES in my current schema by combining COCO_WORKSHOP.SOURCE_DATA.BRONZE_SAP_AP_INVOICES and COCO_WORKSHOP.SOURCE_DATA.BRONZE_ORACLE_AP_INVOICES.

What to look for

• A clean first-pass Dynamic Table definition with explainable design choices.

When finished we can exit plan mode.

Step 1.4 – Use the bundled Dynamic-Table skill

Skills are reusable workflows that tell Cortex Code how to handle a specific Snowflake task. Instead of responding in a completely open-ended way, a skill provides:

- domain context
- expected inputs
- a defined process
- structured outputs

Each skill is packaged as a small folder with a `SKILL.md` file. That file defines what the skill is for, what information it expects, what steps it should follow, and what artifacts it should return, such as SQL, plans, checklists, or evaluation results.

Bundled skills

Bundled skills are Snowflake-maintained skills that ship with Cortex Code. They are prebuilt, Snowflake-native workflows designed by Snowflake's product and AI teams, so you can start from proven patterns instead of a blank prompt.

In this section, you'll use a bundled skill. Later, you'll learn how to create custom skills using the same structure so your team can codify its own workflows.

See what skills are available

From inside a Cortex Code session, list the skills available in your environment:

Prompt

/skill list

Inspect the Dynamic Tables skill

Before applying a skill to your own objects, start by asking it to explain itself:

Prompt

What does the $dynamic-tables skill do? Summarize when I should use it, what inputs it expects, and what kinds of output it returns

This helps you understand the skill before you rely on it.

Apply the skill to a real object
Then apply it to SILVER_AP_INVOICES to generate a practical operating runbook.

Prompt

$dynamic-tables
Analyze the Dynamic Table SILVER_AP_INVOICES in my current schema.
Return:
1. The recommended TARGET_LAG choice for this workflow and why
2. SQL to inspect current state, lag, and refresh history
3. The main failure or staleness patterns to watch for
4. A short best-practices checklist for operating this table well

What to look for

• A runbook you would actually keep: a couple of monitoring queries and a concise operating checklist, not generic advice.

Step 1.5 – Save one proof query (optional)

End Demo 1 with a lightweight proof, not an exhaustive test suite. The goal is to have one simple query you can rerun after changes and to show record counts by source in an easy-to-explain way.

Prompt

Give me one concise proof query for SILVER_AP_INVOICES that shows record counts by SOURCE_SYSTEM and is easy to rerun after future changes.

What to look for

- A concise query that can be reused after every change.

Save this output: sql/02_silver_ap_invoices_proof.sql

By the end of Demo 1 you have a Silver-grade AP invoices Dynamic Table, an operating runbook generated by a bundled skill, and a simple proof query you can rerun after each change.

Demo 2 – Add Local Context and Productionize the Workflow (30 min)

In the first demo, Cortex Code helped you move from natural-language instructions to SQL and an operational view of a Snowflake object. In this demo, you extend that workflow with business context from a local PRD and then turn the pattern into a reusable team asset.

Scenario

A month later, the business sends a PRD that expands the AP invoices pipeline. New source systems must be added, field requirements have changed, and some definitions need clarification.

This is a common kind of request. Rather than solving it once with a long prompt every time, you can standardize the workflow as a custom skill.

Step 2.1 – Read the local PRD

Start by understanding the business request before you design the skill. Assume you have a file such as `sample_business_requirements.xlsx` in your working directory.

Prompt

Read the local file sample_business_requirements.xlsx.
Summarize the changes that affect the AP invoices pipeline.
Return:
- New source systems being introduced
- New fields or business rules that affect the Silver layer
- Ambiguities or open questions that should be resolved before implementation

What to look for

- A clear distinction between requirements and assumptions.
- The shape of the information your custom skill should standardize.

Why Create a Custom Skill Here?

If you stop here and simply ask Cortex Code to update `SILVER_AP_INVOICES`, you can get a reasonable one-off result.

But the repeatable pattern is the real value:

1. read the PRD
2. extract requested changes
3. translate those changes into a Dynamic Table plan
4. surface assumptions and open questions
5. propose validation queries

That is exactly the kind of workflow custom skills are meant to standardize.

What is a Custom Skill

A custom skill is a reusable workflow you define for Cortex Code. In practice, it is usually a small folder containing a `SKILL.md` file that tells Cortex Code:

- when to use the skill
- what inputs it expects
- what steps it should follow
- what outputs it should always return

For this demo, the goal is to create a skill that consistently turns a PRD-style file into an engineering plan for a target Dynamic Table.

Where do custom skills live?

Cortex Code can discover skills from multiple locations:

Skill Type	Location	Scope
Bundled	Built into Cortex Code	Available by default
User-level	`~/.snowflake/cortex/skills/` or `~/.cortex/skills/`	Available across projects
Project-level	`.cortex/skills/` in your repo	Available only in that project

Precedence: project-level > user-level > bundled

For this quickstart, use a **project skill** so anyone who clones the repo gets the same behavior.

Demo 2.2 - Scaffolding a Custom Skill

You can author the skill yourself, but Cortex Code also includes a bundled workflow to help scaffold new skills.

Start by confirming it is available:

Prompt

/skill list

Then ask the skill-development workflow to help you define the new custom skill.

Example Prompt

This seems like a repeatable workflow I will have for many PRDs. Walk me through $skill-development for building a project skill that will help me take PRD-style files like this and turn them into a plan for putting them into a target Dynamic Table (for this demo, SILVER_AP_INVOICES).

Define:
- When to use the skill
- What inputs it expects (for example, prd_path and target_dynamic_table)
- The exact outputs it should always return
- Best practices for surfacing assumptions and open questions instead of guessing
- An example usage for an AP invoices pipeline update

Requirements:
- Make it a project skill
- Put it under .cortex/skills/ in this demo repo
- Start by supporting XLSX files

What this skill should standardize

Your PRD evaluator skill should return the same categories of output every time, such as:

- requested changes
- source-to-Silver mapping
- open questions and assumptions
- DDL delta plan
- validation queries

That consistency is what makes the workflow reusable across teammates and future PRDs.

Best practices for reliable custom skills

When designing a custom skill, keep it narrow and predictable.

- Give it one clear job
- Make the output structure repeatable

- Surface assumptions explicitly instead of silently guessing
- Keep it project-local when it depends on project conventions or objects

In this case, the job is very specific: translate a PRD-like file into a change plan for a target Dynamic Table.

Step 2.4 – Run the PRD Evaluator skill

With the custom skill in place, invoke the workflow instead of rebuilding the logic from scratch. This is the step that turns a one-time prompt into a repeatable team asset.

Prompt

```
Run the project skill we just made prd-to-dt-plan
Context:
- prd_path: sample_business_requirements.xlsx
- target_dynamic_table: SILVER_AP_INVOICES
Return:
1. Summary of requested changes
2. Source-to-Silver mapping summary
3. Open questions and assumptions
4. DDL delta plan for SILVER_AP_INVOICES
5. Validation queries to run after implementation
```

What to look for

• A consistent shape you could compare across future PRDs.

• A delta plan another engineer could review and challenge before deployment.

Save this output: notes/02_prd_change_plan.md

Step 2.5 – Apply the update with Snowflake best practices

Now use the structured output from the skill to update the Dynamic Table. The engineering work is driven by both platform context (existing Snowflake objects) and external business context (the local PRD).

Prompt

Update SILVER_AP_INVOICES in my current schema using the change plan from sample_business_requirements.xlsx.
Assume the PRD introduces Baan and Workday as additional AP invoice sources.
Return:
- The updated Dynamic Table SQL
- A short explanation of how the new sources map into the common schema
- Any assumptions that require engineering review
- Validation queries that prove the update worked

What to look for

• Updated SQL, explainable source mapping, explicit assumptions, and validation queries.

Save this output: sql/03_silver_ap_invoices_prd_update.sql

Step 2.6 – Optional: save the handoff artifacts

If you are treating this as a real project rather than just a lab, finish by saving the change plan, validation queries, and a short note explaining what changed and why. In a Git-backed project, these files live alongside your SQL so another engineer can pull the repo, rerun the checks locally, and see exactly how the PRD was applied.

Prompt

List the artifacts in a local markdown fileI should save from this PRD-driven update so another engineer can review the change, rerun the checks, and reuse the PRD Evaluator skill.

What to look for

• A concise handoff package another engineer can review and rerun.

Save this output: notes/03_prd_workflow_handoff.md

At this point, the quickstart is complete for most teams: you have a curated AP invoices object and a repeatable workflow for evolving it with new PRDs.

Demo 3 (Optional) – Build an Agent on Top of the Finished Data Product (30 minutes)

Why this is optional: the quickstart is complete after Demo 2. Demo 3 is for teams that want to show what comes next: how the governed AP invoices data product you just built can support a focused Cortex data agent that answers finance questions over trusted data rather than querying raw or ad hoc tables.

Client story: By this point you have a curated AP invoices Silver object and a repeatable way to evolve it based on business requirements. That is the right moment to talk about agents, because you can keep the AI experience grounded in trusted, well-modeled data.

Step 3.1 – Define the agent use case

Keep the first pass narrow and grounded in the data product you built. The goal is to make the agent credible, not broad.

Prompt

Help me define a Cortex data agent on top of SILVER_AP_INVOICES in my current schema.
Suggest:
- The primary audience
- The top five business questions the agent should answer
- Guardrails that keep the agent grounded in the curated data
- Any semantic descriptions that would improve answer quality

Step 3.2 – Create a semantic view over the Silver data

Create a semantic view that exposes business-friendly dimensions and measures. This is the object the agent will rely on for most of its answers.

Prompt

Let’s start by building the semantic model using the $semantic-view
Create a semantic view called SV_AP_ANALYTICS over <YOUR_SCHEMA>.SILVER_AP_INVOICES.
It should support natural language questions like:
- "Total AP spend by vendor over the last 12 months"
- "Invoice count by month and business unit"
- "Top 10 vendors by unpaid invoice amount"
Return:
- A complete semantic view definition that clearly names business measures and dimensions.
- Any assumptions you are making about grain, time dimensions, and vendor identifiers.

What to look for

• A semantic view definition with clear business measures, dimensions, and assumptions.

Save this output: sql/semantic/sv_ap_analytics.sql

Step 3.3 – Create a Cortex agent on the semantic view

Now create a Cortex data agent that uses the semantic view to answer natural-language questions. Keep the agent grounded in SV_AP_ANALYTICS and make its answers verifiable.

Prompt

$cortex-agent
Create an agent named AP_ANALYTICS_ASSISTANT.
The agent should:
- Prefer SV_AP_ANALYTICS as its primary data source.
- Always respond with three parts: (1) the final answer, (2) the SQL used, and (3) any assumptions about grain or filters.
- Ask a clarifying question if the requested metric or time grain is ambiguous.
Return a configuration I can save alongside my project files.

What to look for

• A saved configuration that keeps the agent grounded and its outputs verifiable.

Save this output: agents/ap_analytics_assistant.md

Step 3.4 – Evaluate and iterate on the agent

With the agent created, show how teams typically evaluate and iterate. In practice, this often starts with a small golden dataset of stakeholder questions and expected answers.

Prompt

Using Cortex Agent optimize skill, create an evaluation on the golden question dataset our team has set up in a Snowflake table.

What to look for

• A practical evaluation workflow you can operationalize and later turn into a custom skill.

Save this output: notes/04_agent_evaluation_plan.md

Step 3.5 - Cortex Agent Skills and Workflows From Here

Look at all the skills that exist in cortex-agent-optimize

What You Should Take Away

The core pattern is simple: pick one concrete object, ask Cortex Code for one concrete artifact, and keep the result in the project. In Demo 1, that means a Dynamic Table, a small runbook from a bundled skill, and a single proof query you can rerun after every change. In Demo 2, it means treating the PRD and its evaluator as part of the same data product, with a custom skill that turns a loosely written requirements file into a structured, reviewable change plan. By the time you reach the optional agent design in Demo 3, you can see how bundled skills and custom skills together create a path from disciplined data engineering to a credible AI experience.